

CIF Microarchitecture Specification

Block: Read Transaction Table (RTT)

CIF Design Team

June 18, 2026

Contents

1	Overview	2
2	Design Parameters	2
3	Internal Blocks	2
3.1	RTT Register File	2
3.2	Alloc Write Logic	2
3.3	Completion Update Logic	3
3.4	Compare Logic	3
3.5	Free Logic	3
4	Interface Signals	3
4.1	Inputs	3
4.2	Outputs	4
5	Internal Signal Table	4
6	Functional Behaviour	4
6.1	Allocation Flow	4
6.2	Completion Flow	4
6.3	Free Flow	4
6.4	rob_last Routing	5
7	Timing Notes	5
8	Edge Cases	5

1 Overview

The Read Transaction Table (RTT) is a per-transaction tracking structure for the read path. It is allocated at tag assignment time and tracks completion progress for every outstanding read transaction. When all expected packets for a transaction have been received from MC Core, the RTT asserts `rob_last`, signalling the final beat to the ROB and Merge Logic. The entry is freed in the same cycle.

The RTT operates independently of the ROB — both blocks tap the same MC Core completion bus, but serve different purposes: the ROB stores and reorders data; the RTT tracks transaction-level progress and drives `rob_last`.

2 Design Parameters

Parameter	Value	Description
AXID_W	6	AXI ID width
SEQNUM_W	2	Seqnum width
TXN_TAG_W	8	Tag width = AXID_W + SEQNUM_W
RTT_DEPTH	256	Entries = $2^{\text{AXID_W} + \text{SEQNUM_W}}$
EXP_PKTS_W	8	Expected packets width (covers ARLEN+1, max 256)
CMP_PKTS_W	8	Completed packets counter width

Table 1: RTT parameters

3 Internal Blocks

3.1 RTT Register File

- **Structure:** 256-entry register file. Each entry stores:
 - AXID[5:0] — AXI ID of the transaction
 - `exp_pkts`[7:0] — expected packet count (= ARLEN+1)
 - `cmp_pkts`[7:0] — completed packet count (incremented on each MC Core completion)
- **Addressing:** `addr`[7:0] = `TXN_TAG`[7:0] = {AXID[5:0], `seqnum`[1:0]}. The tag directly indexes the register file — no translation required.
- **Write ports:**
 - Alloc Write Logic writes `exp_pkts` on allocation.
 - Completion Update Logic increments `cmp_pkts` on each RD_DATA completion.
 - Free Logic clears the entry (zeroes `cmp_pkts`, invalidates) on `rob_last`.
- **Read port:** Continuously exposes `cmp_pkts` and `exp_pkts` to Compare Logic.

3.2 Alloc Write Logic

- Activated on TAG_VALID from the Tag Allocator.
- Receives `TXN_TAG`[7:0] and `ARLEN`[7:0].

- Computes `exp_pkts = ARLEN + 1`.
- Issues `write_en` to the RTT Register File at address `TXN_TAG[7:0]`, writing `exp_pkts` and initialising `cmp_pkts = 0`.

3.3 Completion Update Logic

- Taps the MC Core completion bus: `RESP_VALID`, `resp_type[1:0]`, `tag[7:0]`.
- **Filter:** acts only when `resp_type == 2'b00` (`RD_DATA`). `WR_ACK` completions (`resp_type == 2'b01`) are destined for the Response Tracker and are ignored. `ERR` completions for read transactions (`resp_type == 2'b10`) are treated as a valid completion event — `cmp_pkts` is still incremented so the transaction reaches `rob_last` and completes normally; error status propagation is handled by the ROB and R Response Generator.
- Asserts `inc_en` to the RTT Register File at address `tag[7:0]`, incrementing `cmp_pkts` by 1.

3.4 Compare Logic

- Purely combinational.
- Reads `cmp_pkts[7:0]` and `exp_pkts[7:0]` from the RTT Register File for the entry currently being updated.
- Asserts `rob_last` as a one-cycle pulse when `cmp_pkts == exp_pkts`.
- `rob_last` is forwarded to two destinations simultaneously: Free Logic (to clear the entry) and the ROB (passed through to Merge Logic coincident with the final data beat).

3.5 Free Logic

- Activated by `rob_last` from Compare Logic.
- Asserts `clear_en` to the RTT Register File at address `tag[7:0]`, clearing the entry in the same cycle as `rob_last`.
- Simultaneously asserts `RD_FREE_VALID` and drives `RD_FREE_TAG[7:0]` to the Tag Allocator's `RD_FREE` port, triggering occupancy decrement for the freed AXID.

4 Interface Signals

4.1 Inputs

Signal	Width	Source	Description
<code>TXN_TAG[7:0]</code>	8	Tag Allocator	Allocated tag; indexes the register file
<code>ARLEN[7:0]</code>	8	Tag Allocator	Burst length; <code>exp_pkts = ARLEN+1</code>
<code>TAG_VALID</code>	1	Tag Allocator	Allocation strobe
<code>RESP_VALID</code>	1	MC Core	Completion valid
<code>resp_type[1:0]</code>	2	MC Core	00= <code>RD_DATA</code> , 01= <code>WR_ACK</code> , 10= <code>ERR</code> ; RTT acts on <code>RD_DATA</code>
<code>tag[7:0]</code>	8	MC Core	Completion tag; used as register file address

Table 2: RTT input signals

4.2 Outputs

Signal	Width	Destination	Description
<code>rob_last</code>	1	ROB / Merge Logic	Pulse on final packet of a transaction
<code>RD_FREE_VALID</code>	1	Tag Allocator (RD_FREE port)	Free strobe; coincident with <code>rob_last</code>
<code>RD_FREE_TAG[7:0]</code>	8	Tag Allocator (RD_FREE port)	Tag being freed

Table 3: RTT output signals

5 Internal Signal Table

Signal	Width	From	To	Description
<code>write_en</code>	1	Alloc Write Logic	RTT Reg File	Write strobe on allocation
<code>wr_addr[7:0]</code>	8	Alloc Write Logic	RTT Reg File	= <code>TXN_TAG[7:0]</code>
<code>exp_pkts[7:0]</code>	8	Alloc Write Logic	RTT Reg File	= <code>ARLEN+1</code>
<code>inc_en</code>	1	Completion Update Logic	RTT Reg File	Increment <code>cmp_pkts</code> strobe
<code>inc_addr[7:0]</code>	8	Completion Update Logic	RTT Reg File	= <code>tag[7:0]</code> from MC Core
<code>cmp_pkts[7:0]</code>	8	RTT Reg File	Compare Logic	Current completed count
<code>exp_pkts[7:0]</code>	8	RTT Reg File	Compare Logic	Expected count (read port)
<code>rob_last</code>	1	Compare Logic	Free Logic, ROB	One-cycle pulse on final completion
<code>clear_en</code>	1	Free Logic	RTT Reg File	Clear entry strobe
<code>clear_addr[7:0]</code>	8	Free Logic	RTT Reg File	= <code>tag[7:0]</code> of completing txn

Table 4: RTT internal signals

6 Functional Behaviour

6.1 Allocation Flow

1. Tag Allocator asserts `TAG_VALID` with `TXN_TAG[7:0]` and `ARLEN[7:0]`.
2. Alloc Write Logic computes `exp_pkts = ARLEN + 1`.
3. RTT Register File is written at address `TXN_TAG[7:0]`: `exp_pkts` is stored, `cmp_pkts` is initialised to zero.

6.2 Completion Flow

1. MC Core asserts `RESP_VALID` with `resp_type == 2'b00` (RD_DATA) and `tag[7:0]`.
2. Completion Update Logic asserts `inc_en` at address `tag[7:0]`: `cmp_pkts` is incremented by 1.
3. Compare Logic reads the updated `cmp_pkts` and `exp_pkts` combinationally.
4. If `cmp_pkts == exp_pkts`: `rob_last` is asserted as a one-cycle pulse.

6.3 Free Flow

1. `rob_last` pulse triggers Free Logic in the same cycle.
2. Free Logic asserts `clear_en` to the RTT Register File: the entry is invalidated.

3. Free Logic simultaneously asserts `RD_FREE_VALID` with `RD_FREE_TAG` to the Tag Allocator, decrementing the occupancy counter for the freed AXID.

6.4 rob_last Routing

`rob_last` is a sideband signal to Merge Logic. It travels in parallel with, not through, the ROB. The ROB emits `data[511:0]`, `status[3:0]`, and `AXID[5:0]` to Merge Logic; the RTT emits `rob_last` to Merge Logic coincident with the final data beat. Merge Logic combines both to assert `RLAST` on the AXI R channel. There is no feedback path from the RTT into the ROB's internal logic.

7 Timing Notes

- Alloc Write Logic is registered: the register file write takes effect on the cycle following `TAG_VALID`.
- Completion Update Logic is registered: `cmp_pkts` increment takes effect on the cycle following `RESP_VALID`.
- Compare Logic is combinational: `rob_last` is available in the same cycle as the `cmp_pkts` update is visible. In practice this means `rob_last` appears one cycle after the final `RESP_VALID`.
- Free Logic fires in the same cycle as `rob_last`: the register file entry is cleared and `RD_FREE_VALID` is asserted without additional latency.
- Simultaneous allocation and completion on the same tag are not possible by construction — a tag is only allocated after its previous occupancy slot is freed, and a completion can only arrive for an in-flight tag.

8 Edge Cases

- **Single-beat transaction (`ARLEN = 0`):** `exp_pkts = 1`. `rob_last` is asserted on the first and only completion. Handled identically to multi-beat transactions.
- **Error completion:** `resp_type == 2'b10 (ERR)` for a read transaction is treated as a valid completion event. `cmp_pkts` is incremented normally so the transaction progresses to `rob_last`. Error status is stored in the ROB slot and forwarded to the R Response Generator as `DECERR`. The RTT does not need to distinguish error from normal completions.
- **WR_ACK completions:** `resp_type == 2'b01` completions are ignored by the RTT entirely. They are processed by the Response Tracker on the write path.
- **Over-completion protection:** `cmp_pkts` must never exceed `exp_pkts`. This is guaranteed by construction since MC Core returns exactly one completion per issued read packet. A design-time assertion should be included in the verification environment.